Mettre à disposition un modèle par le biais d'une API

Lino Galiana
Version du 20 octobre 2026

Les API sont un moyen très pratique pour simplifier la mise à disposition des prédictions d'un modèle de *machine learning*. Ce chapitre présente la manière dont `FastAPI` simplifie l'intégration d'un modèle de machine learning dans une application.

⚠️ **PDF généré automatiquement.** Ce document est produit à partir du site du cours et peut présenter des problèmes de mise en forme. Pour une version à jour et adaptée à la lecture, consultez la page du cours : pythonds.linogaliana.fr.

Table des matières

1 Exemple de réutilisation d'un modèle sous forme d'API **1**
2 Etape 1: créer une application en local **2**
3 Etape 2: créer une API en local **3**
4 Aller plus loin: mettre à disposition cette API de manière pérenne **4**
Informations additionnelles 4
Bibliographie **6**

Ce chapitre présente la deuxième application d'une journée de cours que j'ai donné à l'Université Dauphine dans le cadre des *PSL Data Week*.

L'objectif de ce chapitre est d'amener à développer une API du type de celle-ci.

Dérouler les *slides* associées ci-dessous ou cliquer ici pour les afficher en plein écran.

Le chapitre précédent constituait une introduction à la création de *pipelines* de *machine learning*. Ce chapitre va aller plus loin en montrant la démarche pour le rendre disponible à plus grande échelle par le biais d'une API pouvant être consommée avec de nouvelles données. L'objectif de celle-ci est de ne pas contraindre les réutilisateurs d'un modèle à disposer d'un environnement technique complexe pour pouvoir utiliser le même modèle que celui entraîné précédemment.

1 Exemple de réutilisation d'un modèle sous forme d'API

Un exemple d'API obtenue à l'issue de ce chapitre est mis à disposition sur https://dvf-simple-api.lab.sspcloud.fr/. La documentation de l'API est disponible ici.

Cette API est utilisable dans plusieurs langages.

En `Python`, par exemple, cela donnera:

```
import requests

pieces_principales = 6
```

```
surface = 50
url = f"https://dvf-simple-api.lab.sspcloud.fr/predict?month=4&nombre_lots=1&code_type_local=2&
nombre_pieces_principales={pieces_principales}&surface={surface}"
requests.get(url).json()
```

```
728358.5461884077
```

Néanmoins, l'un des intérêts de proposer une API est que les utilisateurs du modèle ne sont pas obligés d'être des pythonistes. Cela accroît grandement la cible des ré-utilisateurs potentiels.

Cette approche ouvre notamment la possibilité de faire des applications interactives qui utilisent, en arrière plan, notre modèle entraîné avec `Python`.

2 Etape 1: créer une application en local

Mettre en place une API consiste à gravir une marche dans l'échelle de la reproductibilité par rapport à fournir un *notebook*. Ces derniers ne sont pas les outils les plus adaptés pour partager autre chose que du code, à faire tourner de son côté.

Il est donc naturel de sortir des *notebooks* lorsqu'on commence à aller vers ce niveau de mise à disposition. Par le biais de scripts `Python` lancés en ligne de commande, construits en exportant le code du chapitre précédent de nos notebooks, on pourra créer une base de départ propre.

Il est plus naturel de privilégier une interface de développement généraliste comme VSCode à Jupyter lorsqu'on franchit ce rubicon. L'exercice suivant permettra donc de créer cette première application minimale, à exécuter en ligne de commande.

Exercice 1: créer des scripts pour entraîner le modèle

Le dépôt `Github` qui permet de construire l'API *from scratch* est disponible ici. Nous allons emprunter quelques éléments, par-ci par-là, pour faire notre application en local.

- Créer un nouveau service `VSCode` sur le `SSPCloud` en paramétrant dans l'onglet `Networking` le port 5000 ;
- Utiliser la commande suivante depuis le terminal:

```
mkdir app
cd app
```

Depuis le menu des fichiers, créer quatre fichiers dont le contenu suit:

- `requirements.txt`: récupérer le contenu sur cette page ;
- `getdvf.py`: récupérer le contenu sur cette page ;
- `train.py`: récupérer le contenu sur cette page ;
- `api.py`: récupérer le contenu sur cette page.
- Exécuter `getdvf.py` puis `train.py` pour stocker en local le modèle entraîné
- Ajouter `model.joblib` au `.gitignore` *(si vous utilisez `Git`)*
- Créer un script `test.py` qui contient la fonction suivante et la teste après avoir importé votre modèle (`load('pipe.joblib')` en n'oubliant pas `from joblib import load`):

```python
import pandas as pd

def predict(
    month: int = 3,
    nombre_lots: int = 1,
    code_type_local: int = 2,
    nombre_pieces_principales: int = 3,
    surface: float = 75
) -> float:
    """
    """

    df = pd.DataFrame(
        {
            "month": [month],
            "Nombre_de_lots": [nombre_lots],
            "Code_type_local": [code_type_local],
            "Nombre_pieces_principales": [nombre_pieces_principales],
            "surface": [surface]
        }
    )

    prediction = model.predict(df)

    return prediction
```

3 Etape 2: créer une API en local

Le script précédent constitue déjà un progrès dans la reproductibilité. Il rend plus facile le réentraînement d'un modèle sur le même jeu de données. Néanmoins, il reste tributaire du fait que la personne désirant utiliser du modèle utilise `Python` et sache réentrainer le modèle dans les mêmes conditions que vous.

Avec `FastAPI`, nous allons très facilement pouvoir transformer cette application `Python` en une API.

Exercice 2: créer des scripts pour entraîner le modèle

- La ligne ci-dessous du script `api.py` récupère un modèle pré-entraîné enregistré sur un espace de stockage

```
download_file("https://minio.lab.sspcloud.fr/projet-formation/diffusion/python-datascientist/pipe.joblib", 'pipe.joblib')
```

Retirer cette ligne de votre script, pour utiliser le modèle que vous venez d'entraîner.

- Déployer en local l'API avec la commande

```
uvicorn api:app --reload --host "0.0.0.0" --port 5000
```

- A partir du `README` du service VSCode, se rendre sur l'URL de déploiement, ajouter `/docs/` à celui-ci et observer la documentation de l'API
- Se servir de la documentation pour tester les requêtes `/predict`

- Récupérer l'URL d'une des requêtes proposées. La tester dans le navigateur et depuis `Python` avec `Requests` (`requests.get(url).json()`)
- Optionnel: faire tourner le même code dans un autre environnement que le SSPCloud (par exemple une installation de `Python` en local) pour voir que ça fonctionne de manière identique.

4 Aller plus loin: mettre à disposition cette API de manière pérenne

L'étape précédente permettait de créer un point d'accès à votre modèle depuis n'importe quel type de client. A chaque requête de l'API, le script `api.py` était exécuté et renvoyait son *output*.

Ceci est déjà un saut de géant dans l'échelle de la reproductibilité. Néanmoins, cela reste artisanal: si votre serveur local connait un problème (par exemple, vous *killez* l'application), les clients ne recevront plus de réponse, sans comprendre pourquoi.

Il est donc plus fiable de mettre en production sur des serveurs dédiés, qui tournent 24h/24 et qui peuvent également se répartir la charge de travail s'il y a beaucoup de demandes instantanées.

Ceci dépasse néanmoins le cadre de ce cours et sera l'objet d'un cours dédié en 3e année de l'ENSAE: « Mise en production de projets *data science* » donné par Romain Avouac et moi.

Informations additionnelles

i Environnement Python

Ce site a été construit automatiquement par le biais d'une action `Github` utilisant le logiciel de publication reproductible `Quarto` (version 1.10.18).

L'environnement utilisé pour obtenir les résultats est reproductible par le biais d'uv. Le fichier `pyproject.toml` utilisé pour construire cet environnement est disponible sur le dépôt `linogaliana/python-datascientist`

```
[project]
name = "python-datascientist"
version = "0.1.0"
description = "Source code for Lino Galiana's Python for data science course"
readme = "README.md"
requires-python = "⩾3.13,<3.14"
dependencies = [
    "altair⩾6.0.0",
    "cartiflette⩾0.2",
    "contextily==1.6.2",
    "duckdb⩾0.10.1",
    "folium⩾0.19.6",
    "gdal==3.11.4",
    "graphviz==0.20.3",
    "great-tables⩾0.12.0",
    "gt-extras⩾0.0.8",
    "ipykernel⩾6.29.5",
```

```
    "jupyter>=1.1.1",
    "jupyter-cache>=1.0.0",
    "kaleido>=0.2.1",
    "langchain-community>=0.3.27",
    "loguru==0.7.3",
    "markdown>=3.8",
    "nbclient>=0.10.0",
    "nbformat>=5.10.4",
    "nltk>=3.9.1",
    "openpyxl>=3.1.5",
    "pandas>=3.0",
    "pip>=25.1.1",
    "plotly>=6.1.2",
    "plotnine>=0.15",
    "polars>=1.8.2",
    "py7zr>=1.1.3",
    "pyarrow>=17.0.0",
    "pynsee>=0.1.8",
    "python-dotenv>=1.0.1",
    "python-frontmatter>=1.1.0",
    "pywaffle>=1.1.1",
    "requests>=2.32.3",
    "scikit-image>=0.24.0",
    "scikit-learn>=1.8.0",
    "scipy>=1.13.0",
    "seaborn>=0.13.2",
    "selenium<4.39.0",
    "spacy>=3.8.4",
    "webdriver-manager>=4.0.2",
    "wordcloud==1.9.3",
]

[tool.uv.sources]
gdal = [
  { index = "gdal-wheels", marker = "sys_platform == 'linux'" },
  { index = "geospatial_wheels", marker = "sys_platform == 'win32'" },
]

[[tool.uv.index]]
name = "geospatial_wheels"
url = "https://nathanjmcdougall.github.io/geospatial-wheels-index/"
explicit = true

[[tool.uv.index]]
name = "gdal-wheels"
url = "https://gitlab.com/api/v4/projects/61637378/packages/pypi/simple"
explicit = true

[dependency-groups]
dev = [
    "nb-clean>=4.0.1",
]
```

Pour utiliser exactement le même environnement (version de `Python` et *packages*), se reporter à la documentation d'`uv`.

Bibliographie